Guide to the `agent` and `code_sense` Pipelines

> Folders covered:
>
> - `pipelines/agent/` : the AI agent system that writes, edits, checks and reviews documents.
> - `pipelines/code_sense/` : the system that reads and parses source code, builds graphs and reports, and generates BRD and TSD documents.

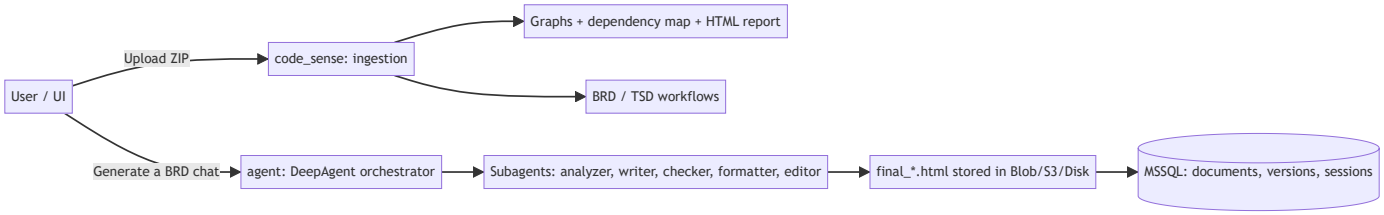
---

0. The big picture

The **ReverseEngineering** product takes a legacy codebase (COBOL, Java, Python, JCL and others) and turns it into documents people can read: BRDs, TSDs, HLDs, call graphs and reports.

There are two engines:

Engine	What it does	Main technology
code_sense	Scans the code, finds languages, functions and dependencies, builds call graphs, writes an HTML analysis report, and generates BRD and TSD documents with a fixed LangGraph workflow.	LangGraph `StateGraph`, language parsers, Kuzu graph DB
agent	An AI team (one orchestrator and several subagents) that reads the code itself, plans the document, writes it section by section, scores it and cleans it up. Results stream live to the UI.	`deepagents` (`create_deep_agent`), LangGraph, SSE streaming



PART A: `pipelines/agent`

A1. Folder map

File / folder	What it does
`builder.py`	Picks the storage backend, seeds the skill and memory files, and **builds the AI team** (orchestrator plus 5 subagents).
`prompts.py`	The detailed instructions for each AI role.
`memory_templates.py`	Content of the `SKILL.md` and `AGENTS.md` files that the agents read while they work.
`tools.py`	Custom tool `request_plan_approval`, which pauses the agent so the user can approve the plan.
`lexicon_tools.py`	Optional tools that search a Neo4j knowledge graph and pgvector. Only some modules get them.
`routes.py`	All FastAPI endpoints (`/agent/...`).
`streaming.py`	Runs the agent and turns its output into **SSE events** for the UI.
`subagent_stream.py`	Shared translator that converts subagent events into SSE. All pipelines use it.
`event_reducer.py`	Turns the raw event log into one saved chat message, so the chat history still exists if the browser was closed.
`version_store.py`	Database layer for documents, versions, sessions, tasks and locks.
`db_init.py`	Creates the MSSQL tables at startup and marks crashed tasks as orphaned.
`azure_blob_backend/`	Lets the agent use Azure Blob as its file system.
`aws_s3_backend/`	The same, for AWS S3.
`verifier/`	A separate agent team that checks an uploaded document against the code.
`coverage/`	A separate agent team that finds code the document does not cover.
`review/`	A separate agent team that reviews the code for security, performance and maintainability.

A2. Step 1: Application startup

In `main.py`:

1. `init_agent_tables()` creates the tables if they don't exist: `agent_document`, `agent_document_version`, `agent_session`, task tables, the lock table and `module_pipeline_type`.
2. `orphan_running_tasks()` marks any task that was "running" when the server stopped as orphaned.
3. `app.include_router(agent_router)` registers every `/agent/*` endpoint.

A3. Step 2: Choosing where files live (storage backend)

`_build_backend()` in `builder.py` decides where the agent's "virtual disk" is:

`CLOUD_STORAGE_PROVIDER`	Backend	Path prefix
`azure`	`AzureBlobBackend`	`<module_id>/...` in the codebase container
`s3` / `aws`	`S3Backend`	`<module_id>/...` in the bucket
anything else	`FilesystemBackend`	`./workspaces/<module_id>/`

All three backends expose the same file operations (`ls`, `read`, `write`, `edit`, `glob`, `grep`), so the agent doesn't need to know where files are stored.

The agent's virtual disk looks like this:

```
/codebase/                  <- uploaded source code
/code_analysis.json         <- codebase summary written by code-analyzer
/code_analysis_meta.json    <- fingerprint of the codebase
/skills/*/SKILL.md          <- how-to guides for the agents
/global/AGENTS.md           <- global memory
/memory/AGENTS.md           <- project memory (lessons learned)
/docs/<document_id>/
    doc_plan.json           <- planned sections
    final_<type>.html       <- the generated document
    quality_review.json     <- scores per section
```

A4. Step 3: Seeding skills and memory

`_seed_skills_and_memory()`:

- **SKILL.md files** are overwritten on every build, so prompt changes in code take effect right away. Skills: `code-analysis`, `mermaid`, `section-writing`, `quality-review`, `document-editing`, `document-formatting`, `html-export`.
- **AGENTS.md files** are written only if they don't exist yet, because they hold user feedback and lessons learned.

A5. Step 4: Building the AI team

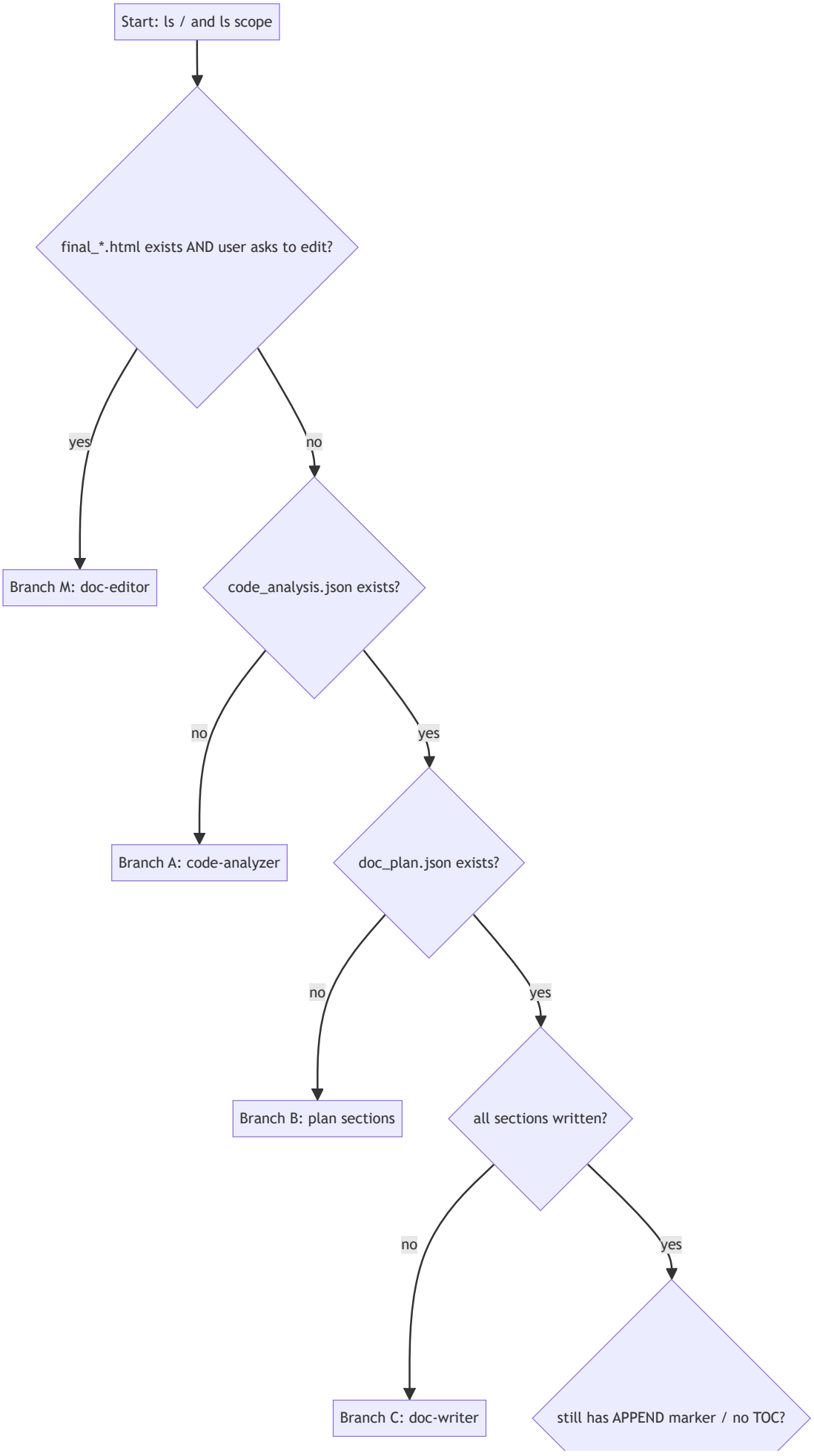
`build_agent()` calls `create_deep_agent(...)` with:

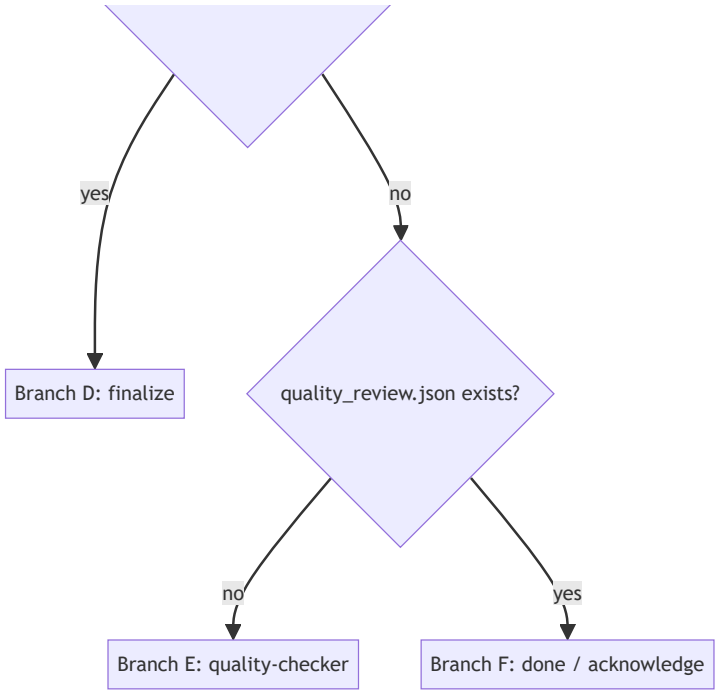
- **Orchestrator** (`ORCHESTRATOR_PROMPT`): the manager. Today's date is inserted so documents show the correct date.
- **Tools:** `request_plan_approval`, plus the Lexicon tools if they are enabled for this module.
- **Checkpointer:** `MemorySaver()`, so the run can pause and resume.
- **5 subagents:**

Subagent	Job	Output
`code-analyzer`	Reads every source file	`/code_analysis.json`
`doc-writer`	Writes each section as HTML into the final document	`<section id=...>` blocks in `final_*.html`
`quality-checker`	Scores each section for completeness, accuracy, clarity and citations	`quality_review.json`
`doc-formatter`	Removes duplicate headings and fixes formatting	Cleaned `final_*.html`
`doc-editor`	Makes targeted edits that the user asks for	Edited `final_*.html`

A6. Step 5: The orchestrator's decision tree

The orchestrator first runs `ls("/")` and `ls("<scope>/")`, then picks **one branch**:


Start: ls / and ls scope
final_*.html exists AND user asks to edit?
yes
no
Branch M: doc-editor
code_analysis.json exists?
no
yes
Branch A: code-analyzer
doc_plan.json exists?
no
yes
Branch B: plan sections
all sections written?
no
yes
Branch C: doc-writer
still has APPEND marker / no TOC?



Because of this tree, an interrupted run **picks up where it stopped** instead of starting over.

A7. Step 6: The full generation pipeline in simple terms

1. **Analyse the code.** `code-analyzer` reads `/codebase/` and writes `/code_analysis.json`.
2. **Plan the sections.** The orchestrator reads the analysis (and the template, if the user uploaded one) and writes `doc_plan.json`.
3. **Ask the user to approve.** `request_plan_approval` calls LangGraph `interrupt()`. The UI shows the plan, and the user can edit it with `PUT /agent/run/{task_id}/plan`. `ResumeCoordinator.signal()` then wakes the agent.
4. **Create the HTML shell.** The orchestrator creates `final_<type>.html` containing a `%%APPEND%%` marker.
5. **Write the sections.** `doc-writer` adds one section at a time by replacing `%%APPEND%%` with `<section>...</section>%%APPEND%%`.
6. **Finalize.** The orchestrator removes `%%APPEND%%`, inserts the table of contents, and checks for duplicate headings and broken mermaid fences.
7. **Gate check (step 7.5).** Any section still missing is retried, up to 3 times.
8. **Quality review.** `quality-checker` writes `quality_review.json`. Sections that score low are rewritten (step 8.5).
9. **Format.** `doc-formatter` cleans up the document.
10. **Update memory.** Sections with weak scores are recorded in `/memory/AGENTS.md`.
11. **Done.** The document is saved as a new version in the database.

A8. Step 7: API flow (`routes.py`)

Starting a run: `POST /agent/run`

Request body: `project_id`, `message`, `history`, `document_id`, `module_id`, `user_id`.

1. Creates a `task_id` (12 hex characters).
2. **Analysis lock:** if `code_analysis.json` doesn't exist yet, `version_store.claim_analysis_lock()` is called. Only **one** document per module can run code analysis at a time. Other requests get **HTTP 409** ("Code analysis is still running...").
3. Sets the document status to `processing`.
4. Converts the chat history into `HumanMessage` / `AIMessage` objects.
5. Creates a `PipelineTask` held in memory and an `agent_task` row in the database.
6. Starts `event_stream(...)` in the background. Events are saved to blob storage regularly (`_persist_event_log`) and a heartbeat is written.
7. Returns the `task_id`.

Other endpoints

Endpoint	Purpose
`GET /agent/run/{task_id}/stream`	SSE live stream. The UI reconnects here.
`GET/PUT /agent/run/{task_id}/plan`	View or edit the plan that is waiting for approval
`POST /agent/run/{task_id}/stop`	Stop a run
`GET /agent/status/{document_id}`	Status of a document's task
`GET /agent/download/{project_id}/{path}`	Download a generated file
`/agent/documents` (GET/POST/PUT/DELETE)	Document CRUD
`/agent/documents/{id}/versions/...`	List, get, update, restore and delete versions, and download a DOCX
`/agent/documents/{id}/template`	Upload, view or delete a custom template (PDF/DOCX/MD)
`/agent/sessions/...`	Chat session CRUD
`POST /agent/verifier/run` + `/stream`	Verifier pipeline
`POST /agent/code-review/run` + `/stream` + `/status`	Code review pipeline
Coverage endpoints	Coverage pipeline

A9. Step 8: Streaming (`streaming.py` + `subagent_stream.py`)

`event_stream()`:

1. Builds a backend and always closes it afterwards (`_close_backend_safely`).
2. Lists `/codebase`. **If there is no code**, it runs a simple chat agent that asks the user to upload code.
3. **Fingerprint check:** computes SHA-256 over (path, size) of all files. If the result differs from `code_analysis_meta.json`, the old analysis is deleted, so a changed codebase is always re-analysed.

4. **Document scope:** adds `[DOCUMENT SCOPE: /docs/{document_id}/]` to the user message so each document keeps its files in its own folder.
5. **Restore:** if `final_*.html` is missing from storage, it is restored from the latest database version.
6. Runs the agent and emits SSE events, including `log`, `token`, `tool_start`, `plan_ready` and `subagent_*`.

`SubagentEventTranslator` turns events from parallel subagents into SSE. Each subagent instance gets a unique `subagent_id`, so the UI keeps their output separate even when two share a name.

A10. Step 9: Saving results (`version_store.py`, `event_reducer.py`)

- Each completed run creates a **new version** (`agent_document_version`).
- Versions can be restored, edited, published or soft-deleted.
- `event_reducer.reduce_run_to_assistant_message()` rebuilds the chat message on the server in the same format the UI uses (`useAgentStream.js`), so the history is saved even if the user closed the tab.

A11. Lexicon tools (optional)

`lexicon_tools.py` adds two read-only tools:

- `lexicon_graph_search`: queries a Neo4j business knowledge graph.
- `lexicon_semantic_search`: runs a vector search with pgvector.

They are enabled **only** for module IDs listed in `Config.LEXICON_ENABLED_MODULE_IDS`. When disabled, both the tools and the prompt text that describes them are left out. The scope is currently hardcoded to a single banking or wealth-management graph.

A12. The three other agent pipelines

All three are built the same way: an orchestrator plus specialist subagents, using the same backend and SSE translator.

Pipeline	Subagents	Output
verifier/	code-indexer, doc-parser, section-annotator, accuracy-auditor, coverage-auditor, report/html assembler, assembly verifier/resumer	Verified HTML with an annotated sidebar, plus a report JSON
coverage/	code-indexer, doc-parser, coverage-auditor, html-renderer	HTML report of code the document misses, with priorities
review/	security-auditor, performance-analyst, maintainability-evaluator, dashboard-compiler	Code review dashboard and report

---

PART B: `pipelines/code_sense`

B1. Folder map

Folder	What it does
`ingestion/`	Unzips the code, detects languages, parses each file with a language analyzer, and runs the main analysis workflow.
`ingestion/analyzers/`	One parser per language (around 20 languages).
`ingestion/preprocessors/`	Cleans COBOL and RPG source before parsing (copybooks, fixed columns).
`graph/`	Builds call and dependency graphs, stores them in Kuzu, detects subsystems and entry points, and powers graph chat.
`report_generators/`	Builds the HTML analysis report (mermaid, tables, CSS/JS).
`doc_generation/`	Fixed LangGraph workflows that generate **BRD** and **TSD** documents.
`trace_analysis.py`	API endpoint that fetches a Langfuse trace and returns a telemetry report.

B2. Step 1: Entry points

In `ingestion/code_analysis.py`:

- `run_code_analysis(zip_path, module_id, project_id)` runs the **full** analysis and report.
- `run_graph_generation(...)` runs the **graph-only** analysis, which is faster.

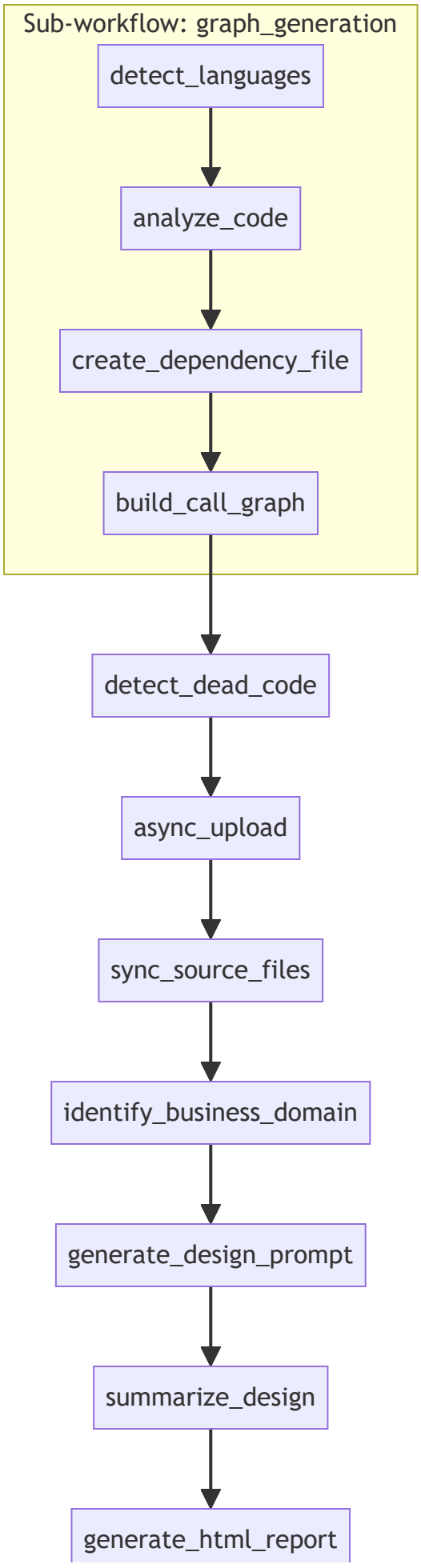
Both:

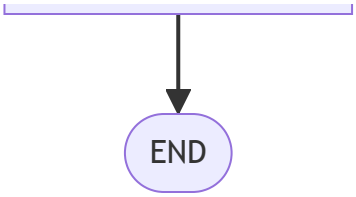
1. Wrap the run in Langfuse tracing (`ensure_trace_user_context`).
2. Unzip into `code_analysis_outputs/<project>/<module>/source`. `extract_nested_zip` also unzips any ZIP files inside the archive.
3. Create a `MultiLanguageCodeAnalysisAgent` and run its workflow.
4. Update the status in the database (`update_ml_analysis_status` / `update_ml_analysis_step`).

The API routes that call these are `api/routes/ml_analysis_reports.py`, `api/routes/graph.py` and `api/routes/embeddings.py`.

B3. Step 2: The LangGraph workflow

All data passes between steps in a shared `AnalysisState` dictionary (`project_path`, `detected_languages`, `analysis_results`, `dependency_map`, `call_graph`, `design_summary`, `html_report_path` and so on).


Sub-workflow: graph_generation
detect_languages
analyze_code
create_dependency_file
build_call_graph
detect_dead_code
async_upload
sync_source_files
identify_business_domain
generate_design_prompt
summarize_design
generate_html_report



Step	What happens
`detect_languages`	`detect_all_languages()` scores each language by file extensions and keywords (`LANGUAGE_INDICATORS`).
`analyze_code`	For each detected language, `MultiLanguageCodeAnalyzerFactory` picks an analyzer, which parses each file into a `CodeAnalysis` object. Runs in parallel threads.
`create_dependency_file`	Combines all dependencies into `multi_language_dependency_map.json`.
`build_call_graph`	`graph_builder.build_cross_language_call_graph()` creates `CallTrace` edges (CALLS, COPIES, PERFORMS, ACCESSES_DB and others) and the data the UI graph needs.
`detect_dead_code`	Finds functions and classes that nothing calls, with line ranges.
`async_upload`	Uploads outputs to Blob or S3 in the background.
`sync_source_files`	Syncs source files to cloud storage in a background thread.
`identify_business_domain`	The LLM works out the business domain (for example banking or insurance).
`generate_design_prompt`	The LLM picks a template (`mainframe`, `web_api`, `database`, `scientific`) and builds a design prompt.
`summarize_design`	The LLM writes the design summary.
`generate_html_report`	`HTMLReportGenerator` + `MermaidProcessor` + `ContentProcessor` produce the HTML report. Mermaid code is repaired, by rules or by the LLM, if it doesn't render.

B4. Step 3: Language analyzers (`ingestion/analyzers/`)

Every analyzer extends `BaseCodeAnalyzer` and returns the same `CodeAnalysis` dataclass:

```
file_path, language, imports, functions, classes, dependencies[DependencyInfo],
entry_points, design_patterns, total_lines, code_lines, comment_lines,
dead_functions, dead_classes, cyclomatic_complexity, function_definition_ranges ...
```

Supported languages: Python, Java, JavaScript, C#, C, C header, C++, COBOL, JCL, BMS, PL/I, RPG, Fortran, Assembly, SQL, XML, JSP, CFML, JD Edwards ER and SabreTalk.

Why this matters: every language produces the same output shape, so all later steps (graphs, reports, documents) work with any language.

B5. Step 4: The graph layer (`graph/`)

File	Simple explanation
`graph_builder.py`	Turns dependencies into nodes (e.g. `COBOL_PROGRAM`, `JAVA_CLASS`) and edges (e.g. `CALLS`, `COPIES`).
`cross_lang_linker.py`	Links code across languages, for example JCL -> COBOL -> DB2 table.
`kg_store.py`	Stores the graph in **Kuzu**, an embedded graph DB, under `data/kg/<project_id>/`. Does nothing if Kuzu isn't installed.
`generic_kuzu_interface.py`, `kuzu_dependency_query.py`	Query helpers for Kuzu.
`graph_service.py`	Loads graph JSON, filters it, gets node details or source code, builds the dependency matrix, and does BFS traversal.
`entrypoint_detector.py`	Finds where programs start (main methods, JCL jobs, routes).
`subsystem_detector.py`	Groups related nodes into subsystems, with LLM-generated names.
`hier_summarizer.py`	Summarizes nodes with the LLM, from the bottom up.
`vector_store.py`	Simple TF-IDF search over graph nodes.
`graph_ai_summary.py` + `extension_questions.py`	"Explain this node", suggested questions, and chat about a node.
`analysis_service.py`	**Dead code.** It imports an `app.*` package that doesn't exist in this repo, so don't use it.

The graph is also used by `pipelines/graph_anchored/` (`graph_traversal.py` imports `KGStore`).

B6. Step 5: Report generators (`report_generators/`)

- `html_report_generator.py` builds the final HTML page.
- `mermaid_processor.py` fixes and renders mermaid diagrams.
- `content_processor.py` cleans up LLM text before it goes into HTML.
- `table_download_utils.py` makes tables downloadable (CSV/Excel).
- `templates/*.txt` holds the prompt templates for each application type, plus `universal_doc_rules.txt`.
- `report_styles.css` and `report_scripts.js` provide the report's styling and interactivity.

B7. Step 6: Document generation (`doc_generation/`)

These are the **older, fixed workflows** for BRD and TSD. Unlike the agent pipeline, the order of steps is fixed in code.

File	Purpose
`generate_brd_template.py` / `generate_tsd_template.py`	A LangGraph workflow that **designs the template** (which sections) from the codebase. BRD steps include classifying the modernization route and discovering business processes and rules.
`app_brd_multi_language.py` / `app_tsd_multi_language.py`	The main generators. They loop over sections.
`brd_generation_for_multi_language.py`	Another BRD workflow that sizes the template to codebase complexity and handles SQL apps.
`llm_validation.py`	Checks each section for completeness and accuracy against the analysis, and fills gaps.
`section_feedback.py`	`apply_section_feedback()` regenerates a single section using user feedback.

How a section loop works (BRD/TSD):

1. `parse_sections_node`: the LLM splits the template into sections and subsections.
2. `determine_section_order_llm` / `classify_sections_batch_llm`: decide the order and type of each section.
3. For each section:
    - `retrieve_chunks_for_section`: RAG retrieval of relevant code chunks plus dependency data.
    - `create_section_content_prompt` -> LLM -> section content.
    - `check_and_fill_gaps` / `llm_validation`: check the section and fill anything missing.
    - `update_master_json` and the component inventory, so later sections know what has already been covered.
    - Rolling summaries keep the LLM context small (`BRD_SUMMARIZATION_CONFIG`).
    - `check_completion` -> `move_to_next_section`.
4. `finalize_documentation` -> `generate_cross_reference_index` -> Markdown/HTML/DOCX export.

PART C: `agent` compared with `code_sense` doc generation

Aspect	`code_sense/doc_generation`	`agent`
Control flow	Fixed LangGraph steps written in code	The LLM orchestrator chooses steps using a decision tree
Code understanding	Pre-parsed analyzers + RAG chunks	The agent reads files itself (`ls` , `grep` , `read_file`)
Document types	BRD, TSD	Any type: BRD, TSD, HLD, SLD, PRD, SAD, API...
User involvement	Feedback on a section after generation	Approves the plan mid-run and edits by chat
Output	Markdown/HTML/DOCX	Single HTML with sections, versioned in the DB
Resume after crash	Limited	Yes (decision tree + event log + DB tasks)
Quality loop	`llm_validation` gap fill	quality-checker scores, then low sections are rewritten

---

PART D: End-to-end example

1. The user uploads `legacy_bank.zip` for module 42.
2. **code_sense** (`run_code_analysis`) unzips it, detects COBOL and JCL, parses the files, builds the call graph, saves it to Kuzu, and produces the HTML report. The UI shows the dependency graph.
3. The user opens Agent Docs and types "*Generate a BRD*".
4. `POST /agent/run` claims the analysis lock and starts `event_stream` .
5. The orchestrator finds no `code_analysis.json` and runs `code-analyzer` .
6. The orchestrator plans the sections and pauses. The UI shows the plan and the user removes one section.
7. `doc-writer` writes each section into `final_brd.html` .
8. The document is finalized with a table of contents, then scored by `quality-checker` . Low sections are rewritten and `doc-formatter` cleans up.
9. A new version is saved to `agent_document_version` and the chat is saved through `event_reducer` .
10. Later the user asks "*Add more detail to Business Rules*". Branch M runs `doc-editor` , which edits only that section and saves a new version.

PART E: Quick glossary

Term	Meaning
DeepAgent	A LangGraph agent with planning (todos), a virtual file system, skills, memory and subagents.
Subagent	A specialist AI that the orchestrator calls through the `task(...)` tool.
Backend	Where the agent's files are stored (disk, Azure Blob or S3).
SSE	Server-Sent Events, the live stream from server to UI.
Interrupt	A LangGraph pause that waits for user input (plan approval).
Fingerprint	A hash of the codebase file list, used to detect code changes.
Kuzu	An embedded graph database used for the code knowledge graph.
Lexicon	An optional business knowledge graph (Neo4j + pgvector).